Rapport de Projet — Science des Données

Prédiction du Churn Client

dans le secteur des Télécommunications

Construction et comparaison de
modèles de *Machine Learning*
pour anticiper la résiliation des clients
et identifier les leviers de rétention

Étudiante :
Rahma Haddouchi
Encadrant :
Naceur Ben Della

Jeu de données :
Telco Customer Churn (Kaggle)
7 043 clients — 21 variables
Outils :
Python, Pandas, Scikit-learn,
XGBoost, SHAP, SMOTE

19 juillet 2026

Table des matières

1 Introduction et contexte **2**
- 1.1 Contexte du projet . . . 2
- 1.2 Objectifs du projet . . . 2
- 1.3 Présentation du jeu de données . . . 2

2 Méthodologie et démarche technique **4**
- 2.1 Étape 0 — Imports et configuration . . . 4
- 2.2 Étape 1 — Chargement et compréhension des données . . . 5
- 2.3 Étape 2 — Vérification des types et des valeurs manquantes . . . 5
- 2.4 Étape 3 — Nettoyage des colonnes numériques . . . 5
- 2.5 Étape 4 — Traitement des valeurs manquantes et des identifiants . . . 5
- 2.6 Analyse exploratoire des données (EDA) . . . 6
  - 2.6.1 Distribution du churn . . . 6
  - 2.6.2 Churn selon le type de contrat . . . 6
  - 2.6.3 Churn selon l'ancienneté (tenure) . . . 7
  - 2.6.4 Charges mensuelles et churn . . . 8
  - 2.6.5 Analyse des corrélations . . . 9
- 2.7 Étape 5 — Préparation des données pour le Machine Learning . . . 10
  - 2.7.1 Encodage des variables catégorielles . . . 10
  - 2.7.2 Séparation en jeux d'entraînement et de test . . . 11
  - 2.7.3 Normalisation des variables numériques . . . 11
  - 2.7.4 Rééquilibrage des classes avec SMOTE . . . 11
- 2.8 Étape 6 — Construction des modèles de classification . . . 12
- 2.9 Étape 7 — Évaluation et comparaison des modèles . . . 13
- 2.10 Étape 8 — Interprétabilité et importance des variables . . . 15
  - 2.10.1 Importance des variables (Feature Importance) . . . 15
  - 2.10.2 Analyse SHAP . . . 16

3 Synthèse des résultats **19**
- 3.1 Comparaison des modèles . . . 19
- 3.2 Facteurs clés du churn . . . 19
- 3.3 Synthèse générale . . . 20

4 Recommandations métier **21**

5 Conclusion **23**

Chapitre 1

Introduction et contexte

1.1 Contexte du projet

Dans un marché des télécommunications fortement concurrentiel, la fidélisation des clients constitue un enjeu stratégique majeur pour les opérateurs. Acquérir un nouveau client coûte en moyenne beaucoup plus cher que d'en conserver un existant : la capacité à anticiper le **churn** (résiliation, attrition) devient donc un avantage compétitif déterminant.

Ce projet a pour ambition de développer un modèle de *Machine Learning* capable de prédire, à partir du profil et du comportement d'un client, le risque qu'il résilie son abonnement, tout en identifiant les facteurs qui influencent le plus cette décision.

1.2 Objectifs du projet

Le projet poursuit sept objectifs précis, qui structurent l'ensemble de la démarche :

O1. **Comprendre les facteurs influençant le churn** — identifier, à travers les données disponibles, les variables susceptibles d'expliquer le départ d'un client.

O2. **Réaliser une analyse exploratoire des données (EDA)** — visualiser et quantifier les relations entre les variables et le churn.

O3. **Préparer les données pour le Machine Learning** — nettoyer, encoder et normaliser le jeu de données afin de le rendre exploitable par des algorithmes supervisés.

O4. **Construire plusieurs modèles de classification** — entraîner des modèles de complexité croissante (régression logistique, arbre de décision, forêt aléatoire, XGBoost).

O5. **Comparer leurs performances** — évaluer chaque modèle selon plusieurs métriques (exactitude, précision, rappel, F1-score, AUC-ROC).

O6. **Identifier les variables les plus importantes** — déterminer, via l'importance des variables et l'analyse SHAP, les leviers d'action les plus pertinents.

O7. **Proposer des recommandations métier** — traduire les résultats analytiques en actions concrètes de rétention client.

1.3 Présentation du jeu de données

Le jeu de données utilisé est le célèbre *Telco Customer Churn*, disponible sur Kaggle. Il rassemble les informations de **7 043 clients** d'un opérateur télécom fictif, décrits par **21 variables**, regroupées en quatre grandes familles :

Catégorie	**Variables**
Informations démographiques	`gender`, `SeniorCitizen`, `Partner`, `Dependents`
Services souscrits	`PhoneService`, `MultipleLines`, `InternetService`, `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, `StreamingMovies`
Informations contractuelles et financières	`tenure`, `Contract`, `PaperlessBilling`, `PaymentMethod`, `MonthlyCharges`, `TotalCharges`
Identifiant et cible	`customerID`, `Churn` (Oui/Non)

Table 1.1: Structure du jeu de données Telco Customer Churn

La variable cible `Churn` présente un déséquilibre notable : environ **26,5 %** des clients (1 869 sur 7 043) ont résilié leur abonnement, contre **73,5 %** qui sont restés. Ce déséquilibre devra être pris en compte lors de l'entraînement des modèles.

Chapitre 2

Méthodologie et démarche technique

Cette section détaille, étape par étape, l'ensemble de la démarche mise en œuvre dans le notebook, depuis le chargement des données jusqu'à l'interprétation des modèles.

2.1 Étape 0 — Imports et configuration

La première étape consiste à importer les bibliothèques nécessaires à chaque phase du projet :

- **Manipulation de données** : `pandas`, `numpy` ;
- **Visualisation** : `matplotlib`, `seaborn` ;
- **Prétraitement** : `train_test_split`, `LabelEncoder`, `StandardScaler` (scikit-learn) ;
- **Modélisation** : `LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier` (scikit-learn), `XGBClassifier` (XGBoost) ;
- **Évaluation** : `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `roc_auc_score`, `roc_curve`, `confusion_matrix`, `classification_report` ;
- **Rééquilibrage des classes** : `SMOTE` (imbalanced-learn) ;
- **Interprétabilité** : `shap`.

Un thème graphique cohérent (`seaborn`, palette `Set2`) est défini pour homogénéiser l'ensemble des visualisations, et une graine aléatoire fixe (`RANDOM_STATE = 42`) est utilisée afin de garantir la reproductibilité des résultats.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, roc_curve,
                             confusion_matrix, classification_report)
from xgboost import XGBClassifier
from imblearn.over_sampling import SMOTE
import shap

```

```
RANDOM_STATE = 42
```

Listing 2.1: Imports et configuration

2.2 Étape 1 — Chargement et compréhension des données

Objectif : charger le fichier `WA_Fn-UseC_-Telco-Customer-Churn.csv` et vérifier ses dimensions ainsi qu'un aperçu de son contenu.

```
df = pd.read_csv('WA_Fn-UseC_-Telco-Customer-Churn.csv')
print("Dimensions :", df.shape)
df.head()
```

Listing 2.2: Chargement du dataset

Cette étape confirme que le jeu de données contient bien 7043 lignes (clients) et 21 colonnes (variables), et permet un premier contact visuel avec la structure des données.

2.3 Étape 2 — Vérification des types et des valeurs manquantes

Objectif : inspecter le schéma du dataset (`df.info()`) afin de repérer les colonnes dont le type nécessite une correction avant modélisation.

```
df.info()
```

Listing 2.3: Inspection du schéma

Cette inspection révèle notamment que la colonne `TotalCharges`, qui représente pourtant une grandeur numérique (montant total facturé), est stockée sous forme de texte (*string*) — un problème classique de qualité des données qu'il convient de corriger.

2.4 Étape 3 — Nettoyage des colonnes numériques

Objectif : convertir `TotalCharges` en type numérique afin qu'elle puisse être utilisée dans les calculs statistiques et les modèles.

```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
print(df.isnull().sum()[df.isnull().sum() > 0])
```

Listing 2.4: Conversion de TotalCharges

Le paramètre `errors='coerce'` transforme automatiquement en valeur manquante (`NaN`) toute chaîne de caractères non convertible en nombre — ce qui permet d'identifier les lignes problématiques.

2.5 Étape 4 — Traitement des valeurs manquantes et des identifiants

Objectif : traiter les valeurs manquantes générées à l'étape précédente et retirer les colonnes non pertinentes pour la modélisation.

```
# Les valeurs manquantes correspondent a des clients avec tenure = 0
# (nouveaux clients, pas encore factures)
df['TotalCharges'] = df['TotalCharges'].fillna(0)
df.drop(columns=['customerID'], inplace=True)
print("Doublons :", df.duplicated().sum())
```

Listing 2.5: Traitement des valeurs manquantes

L'analyse montre que les valeurs manquantes de `TotalCharges` correspondent systématiquement à des clients dont l'ancienneté (`tenure`) est nulle — il s'agit de nouveaux clients qui n'ont pas encore reçu de facturation. Elles sont donc remplacées par 0, ce qui est cohérent avec la réalité métier. La colonne `customerID`, purement identifiante, est supprimée car elle n'apporte aucune information prédictive. Une vérification de doublons est également effectuée.

2.6 Analyse exploratoire des données (EDA)

Objectif : visualiser la distribution du churn et identifier les variables les plus discriminantes, afin de guider la suite de la modélisation.

2.6.1 Distribution du churn

```
churn_rate = df['Churn'].value_counts(normalize=True)['Yes'] * 100
sns.countplot(data=df, x='Churn', hue='Churn',
              palette=['#2ecc71', '#e74c3c'], legend=False)
plt.title('Repartition Churn / Non-Churn')
plt.show()
print(f"Taux de churn : {churn_rate:.1f}%")
```

Listing 2.6: Répartition churn / non-churn

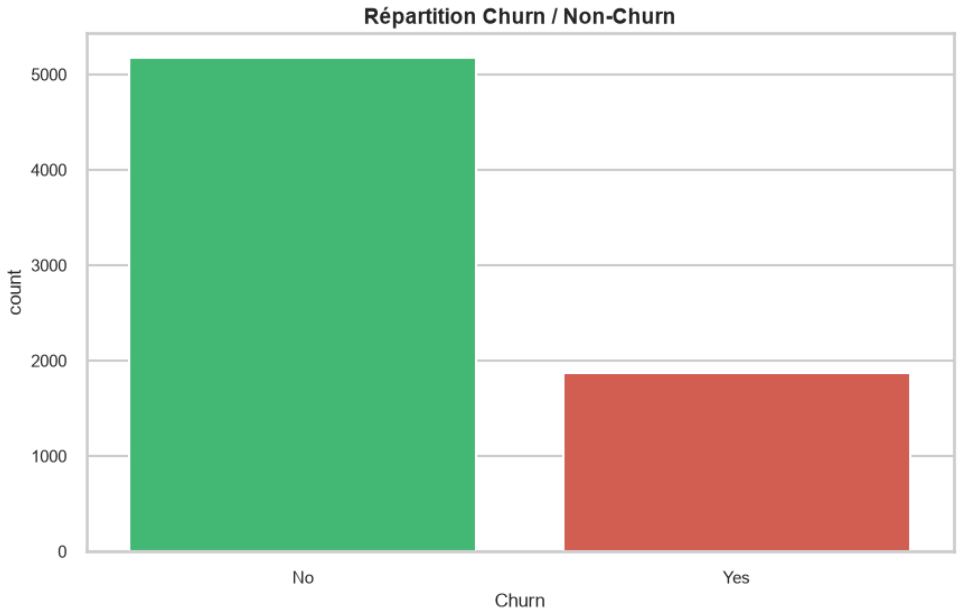


Figure 2.1: Distribution du churn dans le jeu de données

Cette visualisation confirme la structure observée au niveau du dataset : la majeure partie des clients reste fidèle à l'opérateur, tandis qu'une proportion plus faible présente un risque de résiliation. Le graphique montre clairement que le nombre de clients non churneurs est nettement supérieur à celui des clients churneurs, avec un taux d'environ 26,5 % de résiliations. Ce déséquilibre des classes est un élément central à prendre en compte, car il influence directement le choix des méthodes de modélisation et la stratégie de calibration des modèles.

2.6.2 Churn selon le type de contrat

```
sns.countplot(data=df, x='Contract', hue='Churn',
              palette=['#2ecc71', '#e74c3c'])
plt.title('Churn selon le type de contrat')
plt.show()
```

Listing 2.7: Churn selon le contrat

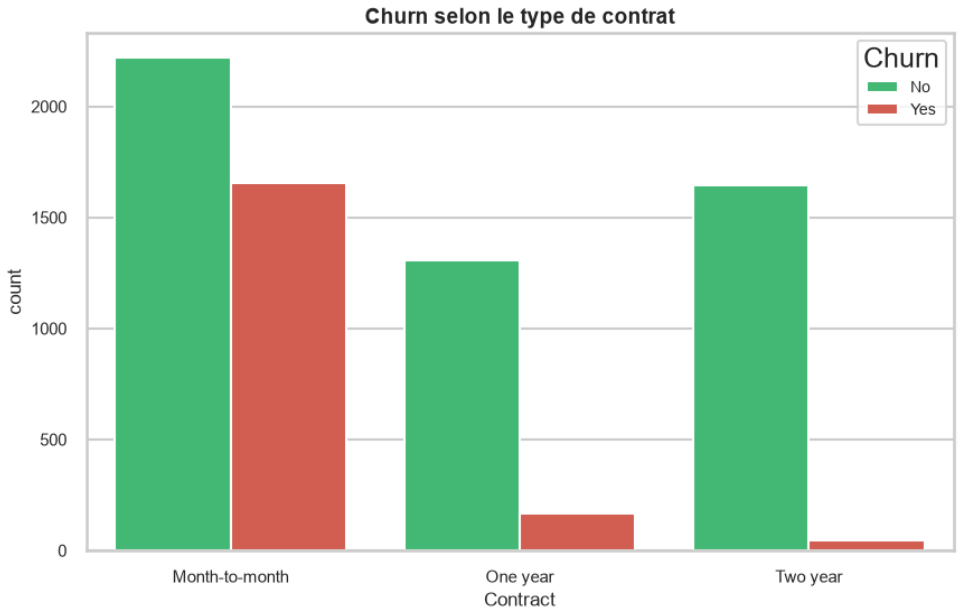


Figure 2.2: Répartition du churn selon le type de contrat

Cette analyse permet de vérifier si les clients engagés sur un contrat court terme (mensuel) résilient davantage que ceux ayant souscrit un engagement annuel ou biannuel. Elle met en évidence l'importance du type de contrat dans la fidélisation : plus un client est engagé sur un contrat à long terme, plus il a tendance à rester fidèle à l'opérateur. À l'inverse, les contrats mensuels sont souvent associés à une plus forte sensibilité au marché et à une probabilité plus élevée de résiliation. Cette observation confirme que le type de contrat constitue un levier stratégique pour les actions de rétention client.

2.6.3 Churn selon l'ancienneté (tenure)

```
sns.histplot(data=df, x='tenure', hue='Churn', multiple='stack',
             palette=['#2ecc71', '#e74c3c'])
plt.title("Churn selon l'anciennete (tenure)")
plt.show()
```

Listing 2.8: Churn selon l'anciennete

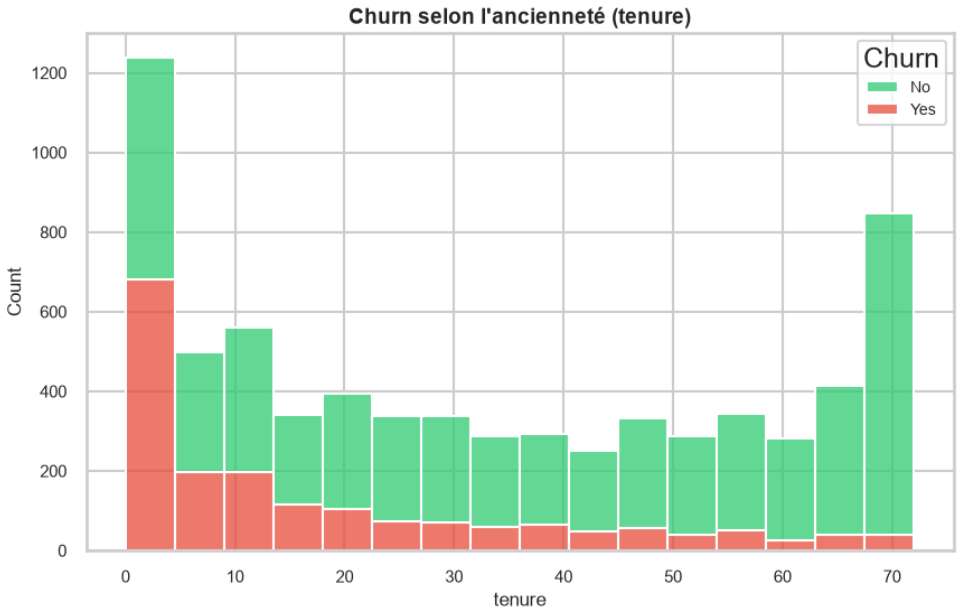


Figure 2.3: Churn selon l'ancienneté (tenure)

Cette visualisation permet d'observer si le risque de churn est concentré chez les clients récents, ce qui orienterait les actions de rétention vers les premiers mois de la relation client. Le graphique confirme très nettement cette hypothèse : la part de clients churneurs (en rouge) est maximale sur les premiers mois d'ancienneté, avec près de 680 résiliations concentrées entre 0 et 5 mois, puis elle décroît de façon quasi continue à mesure que l'ancienneté augmente. Au-delà de 30 mois environ, le nombre de clients churneurs devient marginal comparé au nombre de clients fidèles. On remarque également un pic de clients (essentiellement non-churneurs) autour de 70 mois d'ancienneté, correspondant vraisemblablement à une cohorte de clients très fidèles ayant souscrit depuis longtemps. Ce constat renforce la recommandation R2 : les premiers mois de la relation client représentent la période la plus critique pour la rétention, et méritent un accompagnement renforcé (onboarding, offres de bienvenue, suivi proactif).

2.6.4 Charges mensuelles et churn

```
sns.boxplot(data=df, x='Churn', y='MonthlyCharges', hue='Churn',
            palette=['#2ecc71', '#e74c3c'], legend=False)
plt.title('Charges mensuelles selon churn')
plt.show()
```

Listing 2.9: Charges mensuelles selon le churn

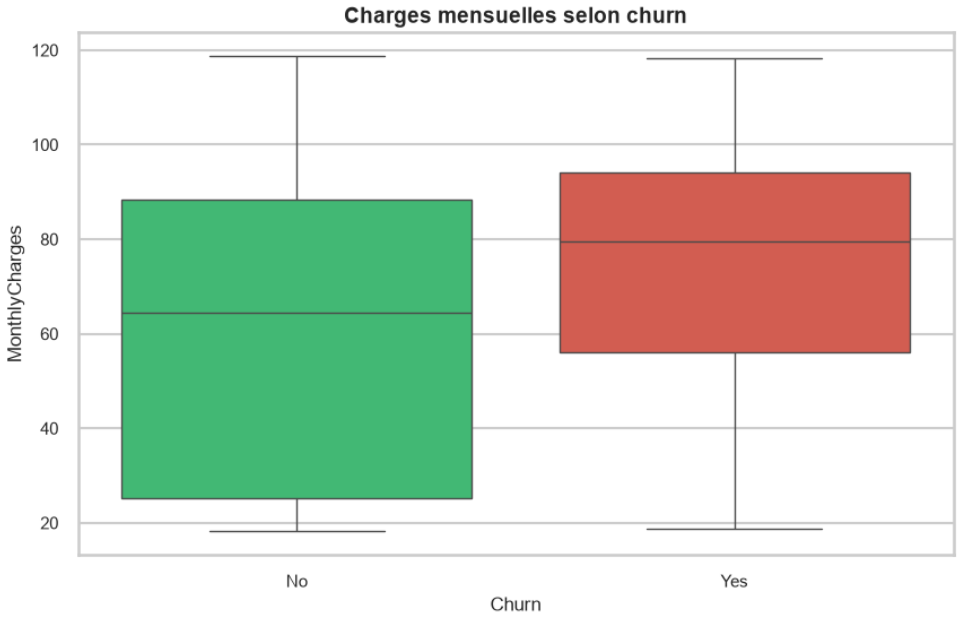


Figure 2.4: Charges mensuelles selon le churn

Cette étape compare la distribution des charges mensuelles entre les deux groupes de clients, afin d'évaluer si un coût plus élevé constitue un facteur de résiliation. Le boxplot met en évidence un écart net entre les deux populations : les clients churneurs affichent une médiane de charges mensuelles proche de 79 $, contre environ 64 $ pour les clients fidèles. La boîte des churneurs est également décalée vers le haut dans son ensemble, avec un premier quartile (˜56 $) supérieur à celui des non-churneurs (˜25 $), tandis que les charges maximales restent comparables entre les deux groupes (autour de 118-120 $). Ce décalage global de distribution suggère que les clients payant des factures mensuelles plus élevées perçoivent davantage un rapport qualité-prix défavorable, ou sont plus sensibles aux offres concurrentes moins chères, ce qui accroît leur propension à résilier. Ce constat appuie la recommandation R4 visant à revoir la politique tarifaire pour les clients les plus exposés.

2.6.5 Analyse des corrélations

```
num_df = df[['tenure', 'MonthlyCharges', 'TotalCharges']].copy()
num_df['Churn_bin'] = df['Churn'].map({'Yes': 1, 'No': 0})
sns.heatmap(num_df.corr(), annot=True, cmap='coolwarm', center=0)
plt.title('Correlation avec le churn')
plt.show()
```

Listing 2.10: Matrice de correlation

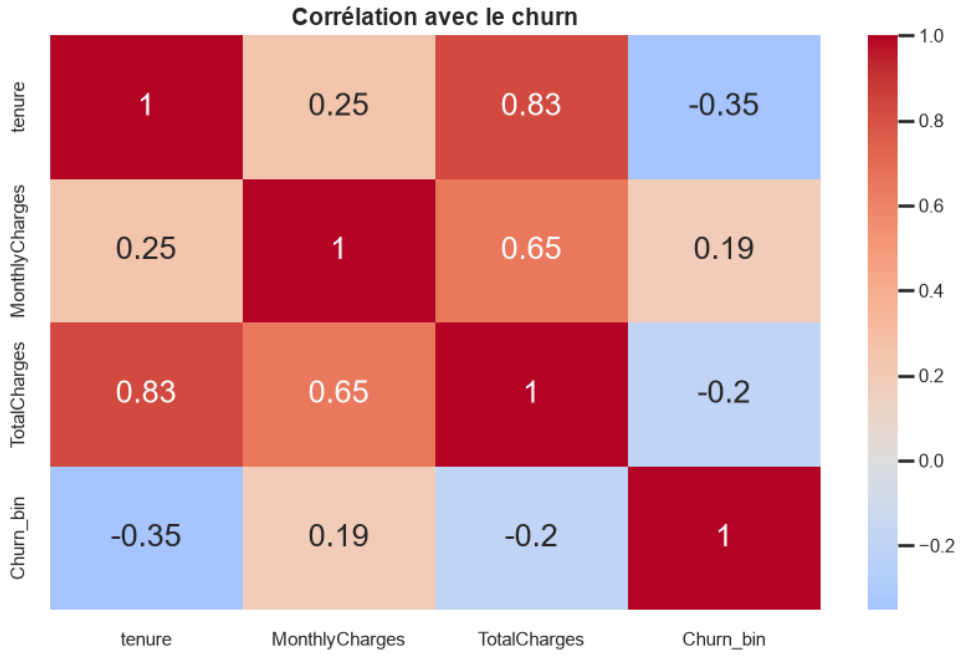


Figure 2.5: Matrice de corrélation avec le churn

Une matrice de corrélation est construite entre les variables numériques (`tenure`, `MonthlyCharges`, `TotalCharges`) et une version binarisée de la cible, afin de quantifier la force et le sens des relations linéaires avec le churn. La lecture de la matrice apporte plusieurs enseignements. D'une part, sans surprise, `tenure` et `TotalCharges` sont fortement corrélées (0,83), puisque le montant total facturé augmente mécaniquement avec l'ancienneté du client ; une corrélation modérée (0,65) existe également entre `MonthlyCharges` et `TotalCharges`. D'autre part, et c'est le point le plus intéressant pour la problématique métier, la variable `tenure` présente la corrélation la plus forte avec le churn parmi les variables numériques, avec un coefficient de $-0,35$: plus un client est ancien, moins il a tendance à résilier, ce qui confirme quantitativement l'observation faite en section 2.6.3. À l'inverse, `MonthlyCharges` est corrélée positivement au churn (0,19) : des charges mensuelles plus élevées sont associées à une propension légèrement plus forte à résilier, cohérent avec l'analyse du boxplot en section 2.6.4. Enfin, `TotalCharges` affiche une corrélation négative modérée avec le churn ($-0,2$), qui s'explique surtout par sa forte dépendance à l'ancienneté plutôt que par un effet direct. Ces corrélations restent modérées, ce qui justifie le recours à des modèles non linéaires (Random Forest, XGBoost) capables de capturer des interactions plus complexes entre variables.

2.7 Étape 5 — Préparation des données pour le Machine Learning

Objectif : transformer le jeu de données brut en une matrice de variables entièrement numériques, exploitable par les algorithmes d'apprentissage supervisé.

2.7.1 Encodage des variables catégorielles

```
df_model = df.copy()
df_model['Churn'] = df_model['Churn'].map({'Yes': 1, 'No': 0})

binary_cols = ['gender', 'Partner', 'Dependents', 'PhoneService',
               'PaperlessBilling']
le = LabelEncoder()
```

```
for col in binary_cols:
    df_model[col] = le.fit_transform(df_model[col])

multi_cols = ['MultipleLines', 'InternetService', 'OnlineSecurity',
              'OnlineBackup', 'DeviceProtection', 'TechSupport',
              'StreamingTV', 'StreamingMovies', 'Contract',
              'PaymentMethod']
df_model = pd.get_dummies(df_model, columns=multi_cols, drop_first=True)
```

Listing 2.11: Encodage des variables

Deux stratégies d'encodage sont combinées :

- un **encodage binaire** (`LabelEncoder`) pour les variables à deux modalités (`gender`, `Partner`, `Dependents`, `PhoneService`, `PaperlessBilling`) ;
- un **encodage one-hot** (`pd.get_dummies`, avec suppression de la première modalité pour éviter la colinéarité) pour les variables catégorielles comportant plus de deux modalités (type de contrat, méthode de paiement, services additionnels, etc.).

La variable cible `Churn` est quant à elle recodée en binaire (1 = résiliation, 0 = maintien).

2.7.2 Séparation en jeux d'entraînement et de test

```
X = df_model.drop(columns=['Churn'])
y = df_model['Churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
```

Listing 2.12: Split train/test

Le jeu de données est scindé en un ensemble d'entraînement (80 %) et un ensemble de test (20 %). Le paramètre `stratify=y` garantit que la proportion de clients churneurs est identique dans les deux sous-ensembles, ce qui est essentiel compte tenu du déséquilibre des classes.

2.7.3 Normalisation des variables numériques

```
num_features = ['tenure', 'MonthlyCharges', 'TotalCharges']
scaler = StandardScaler()

X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()
X_train_scaled[num_features] = scaler.fit_transform(X_train[num_features])
X_test_scaled[num_features] = scaler.transform(X_test[num_features])
```

Listing 2.13: Standardisation

Les trois variables numériques continues (`tenure`, `MonthlyCharges`, `TotalCharges`) sont standardisées (moyenne nulle, écart-type unitaire) via `StandardScaler`. Le *scaler* est ajusté (`fit`) uniquement sur le jeu d'entraînement, puis appliqué (`transform`) aux deux jeux, afin d'éviter toute fuite d'information (*data leakage*) entre l'entraînement et le test.

2.7.4 Rééquilibrage des classes avec SMOTE

```
smote = SMOTE(random_state=RANDOM_STATE)
X_train_bal, y_train_bal = smote.fit_resample(X_train_scaled, y_train)
print("Avant :", y_train.value_counts().to_dict())
print("Apres :", y_train_bal.value_counts().to_dict())
```

Listing 2.14: Reequilibrage SMOTE

La technique **SMOTE** (*Synthetic Minority Over-sampling Technique*) génère des exemples synthétiques de la classe minoritaire (clients churneurs) par interpolation entre voisins proches, afin d'équilibrer les deux classes dans le jeu d'entraînement. Cette étape vise à empêcher les modèles de biaiser leurs prédictions en faveur de la classe majoritaire. **Important :** SMOTE est appliqué uniquement sur le jeu d'entraînement, jamais sur le jeu de test, afin de conserver une évaluation représentative de la réalité.

2.8 Étape 6 — Construction des modèles de classification

Objectif : entraîner plusieurs algorithmes de classification de complexité croissante, afin de comparer leur capacité à prédire le churn.

```
models = {
    'Regression Logistique': LogisticRegression(max_iter=1000,
                                                random_state=RANDOM_STATE),
    'Decision Tree': DecisionTreeClassifier(max_depth=6,
                                            random_state=RANDOM_STATE),
    'Random Forest': RandomForestClassifier(n_estimators=300, max_depth=10,
                                            random_state=RANDOM_STATE),
    'XGBoost': XGBClassifier(n_estimators=300, max_depth=5,
                             learning_rate=0.05, eval_metric='logloss',
                             random_state=RANDOM_STATE)
}

trained_models = {}
for name, model in models.items():
    model.fit(X_train_bal, y_train_bal)
    trained_models[name] = model
```

Listing 2.15: Entrainement des quatre modeles

Quatre familles de modèles sont entraînées sur le jeu équilibré (`X_train_bal`, `y_train_bal`) :

Modèle	**Principe et hyperparamètres**
Régression Logistique	Modèle linéaire de référence, simple et interprétable (`max_iter=1000`).
Arbre de décision	Modèle non linéaire basé sur des règles de séparation successives (`max_depth=6` pour limiter le sur-apprentissage).
Forêt aléatoire (*Random Forest*)	Ensemble de 300 arbres de décision (`max_depth=10`), combinés par vote majoritaire, réduisant la variance.
XGBoost	Algorithme de *gradient boosting* performant, avec 300 estimateurs, une profondeur maximale de 5 et un taux d'apprentissage de 0,05.

Table 2.1: Modèles entraînés et hyperparamètres associés

2.9 Étape 7 — Évaluation et comparaison des modèles

Objectif : mesurer objectivement la performance de chaque modèle sur le jeu de test (non vu pendant l'entraînement) à l'aide de plusieurs métriques complémentaires.

```
results = []
for name, model in trained_models.items():
    y_pred = model.predict(X_test_scaled)
    y_proba = model.predict_proba(X_test_scaled)[:, 1]
    results.append({
        'Modele': name,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1-score': f1_score(y_test, y_pred),
        'ROC-AUC': roc_auc_score(y_test, y_proba)
    })

results_df = pd.DataFrame(results).set_index('Modele').round(3)
results_df.sort_values('ROC-AUC', ascending=False)
```

Listing 2.16: Calcul des metriques de performance

Cinq métriques sont calculées pour chaque modèle :

- **Accuracy (exactitude)** : proportion globale de prédictions correctes ;
- **Precision (précision)** : parmi les clients prédits churneurs, la proportion réellement churneurs ;
- **Recall (rappel)** : parmi les clients réellement churneurs, la proportion correctement détectée — métrique cruciale en contexte de rétention, car manquer un client à risque a un coût métier élevé ;
- **F1-score** : moyenne harmonique entre précision et rappel ;
- **ROC-AUC** : capacité globale du modèle à distinguer les deux classes, indépendamment du seuil de décision choisi.

```
results_df.plot(kind='bar', figsize=(12, 6))
plt.title('Comparaison des performances')
plt.ylim(0, 1)
plt.xticks(rotation=20)
plt.show()
```

Listing 2.17: Graphique comparatif des performances

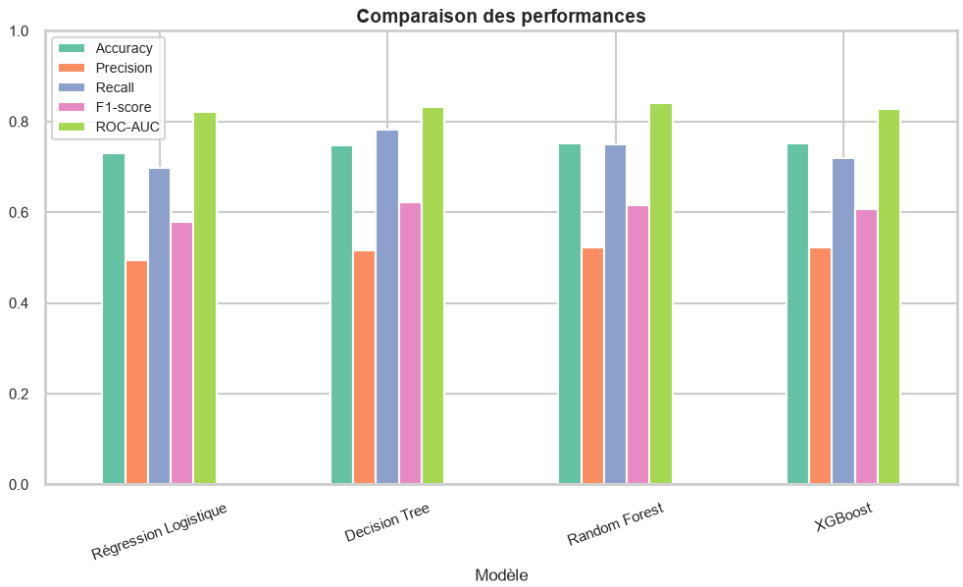


Figure 2.6: Comparaison des performances des modèles

Un graphique en barres permet de comparer visuellement les quatre modèles sur l'ensemble des métriques simultanément.

```
plt.figure(figsize=(8, 7))
for name, model in trained_models.items():
    y_proba = model.predict_proba(X_test_scaled)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    auc = roc_auc_score(y_test, y_proba)
    plt.plot(fpr, tpr, label=f'{name} (AUC={auc:.3f})')
plt.plot([0,1],[0,1],'k--')
plt.legend()
plt.title('Courbes ROC')
plt.show()
```

Listing 2.18: Courbes ROC

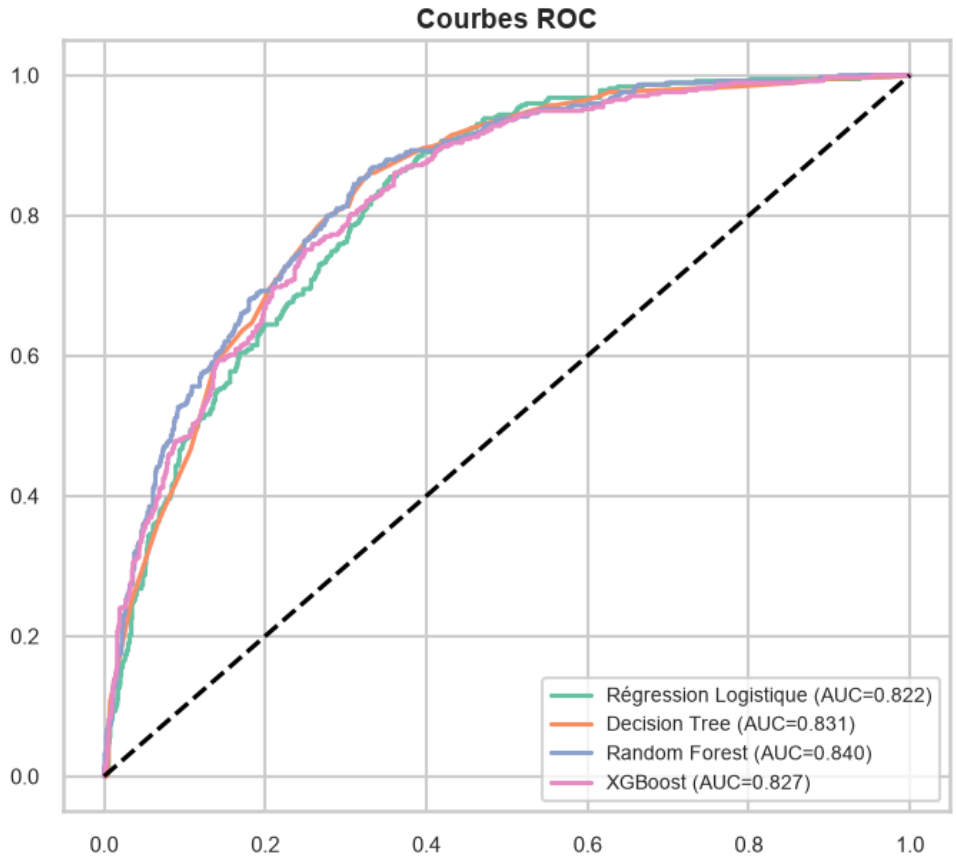


Figure 2.7: Courbes ROC des modèles

Les **courbes ROC** (*Receiver Operating Characteristic*) permettent de visualiser, pour chaque modèle, le compromis entre le taux de vrais positifs et le taux de faux positifs à différents seuils de décision. Plus une courbe se rapproche du coin supérieur gauche, plus le modèle est performant.

```
best_model_name = results_df['ROC-AUC'].idxmax()
best_model = trained_models[best_model_name]
print("Meilleur modele :", best_model_name)
print(classification_report(y_test, best_model.predict(X_test_scaled)))
```

Listing 2.19: Selection du meilleur modele

Le modèle présentant le meilleur score **ROC-AUC** est retenu comme modèle final, et un rapport de classification détaillé (précision, rappel, F1-score par classe) est généré pour ce modèle.

2.10 Étape 8 — Interprétabilité et importance des variables

Objectif : comprendre *pourquoi* le modèle prédit un client comme churneur, afin de transformer les résultats statistiques en recommandations métier actionnables.

2.10.1 Importance des variables (Feature Importance)

```
importances = pd.Series(best_model.feature_importances_,
                        index=X_train.columns)
importances.sort_values(ascending=False).head(15) \
    .sort_values().plot(kind='barh', figsize=(10,8))
plt.title(f'Top 15 variables - {best_model_name}')
plt.show()
```

Listing 2.20: Top 15 des variables importantes

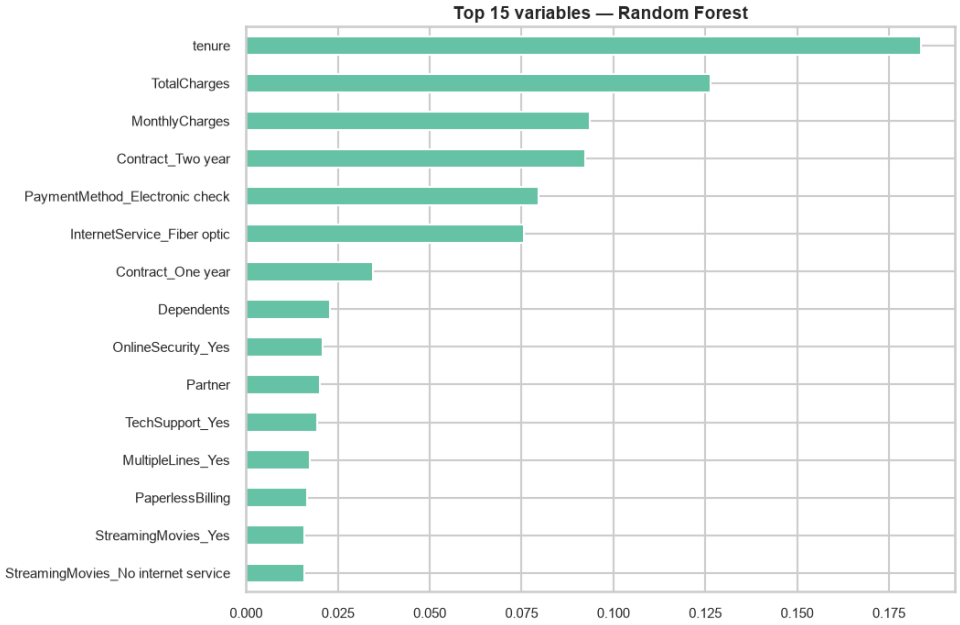


Figure 2.8: Top 15 des variables les plus importantes — Random Forest

Les modèles à base d'arbres (Random Forest, XGBoost) fournissent nativement un score d'importance pour chaque variable, reflétant sa contribution moyenne à la réduction de l'impureté lors des séparations. Les 15 variables les plus influentes sont représentées sous forme de diagramme à barres horizontales. Pour le modèle retenu (Random Forest), la variable `tenure` arrive largement en tête avec un score d'importance proche de 0,185, loin devant `TotalCharges` (˜0,125) et `MonthlyCharges` (˜0,09). Ces trois variables numériques liées à l'ancienneté et à la facturation concentrent, à elles seules, plus de 40 % de l'importance totale du modèle. Viennent ensuite deux variables catégorielles directement liées au contrat : `Contract_Two year` et `PaymentMethod_Electronic check`, suivies de `InternetService_Fiber optic`, qui traduisent respectivement l'effet protecteur des engagements longue durée et l'effet aggravant du paiement par chèque électronique et de l'offre fibre. Les variables purement démographiques (`Dependents`, `Partner`) ainsi que les services annexes (`OnlineSecurity_Yes`, `TechSupport_Yes`, `StreamingMovies_Yes`) occupent des positions plus basses dans le classement, avec des scores d'importance inférieurs à 0,025, confirmant qu'ils jouent un rôle secondaire par rapport aux variables contractuelles et financières.

2.10.2 Analyse SHAP

```
explainer = shap.TreeExplainer(best_model)
X_sample = X_test_scaled.sample(500, random_state=RANDOM_STATE)
shap_values = explainer.shap_values(X_sample)
sv = shap_values[1] if isinstance(shap_values, list) else shap_values
shap.summary_plot(sv, X_sample)
```

Listing 2.21: Analyse SHAP



Figure 2.9: Analyse des interactions SHAP

L'analyse **SHAP** (*SHapley Additive exPlanations*), issue de la théorie des jeux coopératifs, va plus loin que la simple importance des variables : elle quantifie, pour **chaque prédiction individuelle**, la contribution positive ou négative de chaque variable. Le *summary plot* obtenu sur un échantillon de 500 clients test permet de visualiser simultanément :

- le classement global des variables par impact moyen sur la prédiction ;
- le sens de l'effet (une valeur élevée de la variable pousse-t-elle vers le churn ou vers la fidélité ?) ;

- la dispersion de cet effet selon les clients.

Chapitre 3

Synthèse des résultats

3.1 Comparaison des modèles

Les quatre modèles entraînés (régression logistique, arbre de décision, forêt aléatoire, XGBoost) affichent des performances globalement proches, mais avec des écarts significatifs selon la métrique observée. Le tableau ci-dessous synthétise les résultats obtenus sur le jeu de test.

Modèle	Accuracy	Precision	Recall	F1-score	ROC-AUC
Régression Logistique	0,731	0,495	0,698	0,579	0,822
Decision Tree	0,748	0,517	0,783	0,623	0,831
Random Forest	0,752	0,523	0,749	0,616	**0,840**
XGBoost	0,752	0,523	0,719	0,606	0,827

Table 3.1: Comparaison des performances des quatre modèles sur le jeu de test

Le modèle **Random Forest** obtient le meilleur score ROC-AUC (0,840), devançant de peu XGBoost (0,827) et l'arbre de décision seul (0,831). C'est le **Decision Tree** qui offre le meilleur rappel (0,78), c'est-à-dire la meilleure capacité à détecter les clients réellement churneurs — une qualité précieuse dans un contexte de rétention où manquer un client à risque coûte cher. La régression logistique, plus simple et plus interprétable, reste compétitive malgré sa nature linéaire, ce qui confirme que les relations entre variables et churn, bien que non triviales, ne nécessitent pas nécessairement une grande complexité algorithmique pour être en partie capturées. Sur l'ensemble des métriques, l'écart entre les quatre modèles reste modéré (moins de 2 points de ROC-AUC), ce qui suggère que la valeur ajoutée provient surtout de la qualité des variables et de la préparation des données (encodage, SMOTE, normalisation) plutôt que du choix de l'algorithme.

3.2 Facteurs clés du churn

La mise en commun de l'analyse exploratoire (section 2.6), de la matrice de corrélation et de l'analyse d'interprétabilité (importance des variables et SHAP, section 2.10) permet de dégager cinq facteurs déterminants du churn :

- **Ancienneté** (`tenure`) : c'est la variable la plus déterminante du modèle Random Forest, avec un score d'importance de ~0,185, loin devant toutes les autres variables. Le risque de résiliation est concentré sur les premiers mois de la relation client, avec une corrélation négative de $-0,35$ entre l'ancienneté et le churn — plus un client reste longtemps, moins il est susceptible de partir.

- **Charges facturées** (`TotalCharges`, `MonthlyCharges`) : ces deux variables arrivent en deuxième et troisième position dans le classement d'importance (~0,125 et ~0,09). Les clients churneurs paient en médiane environ 79 $ par mois contre 64 $ pour les clients fidèles, avec une corrélation positive de 0,19 entre `MonthlyCharges` et le churn : des factures plus élevées augmentent la propension à résilier.
- **Type de contrat** (`Contract`) : les clients en contrat mensuel (*Month-to-month*) représentent la quasi-totalité des résiliations observées, contre une minorité pour les contrats engagés sur un an ou deux ans. La modalité `Contract_Two year` figure au 4e rang des variables les plus importantes, confirmant l'effet protecteur des engagements longue durée.
- **Méthode de paiement et type d'accès internet** (`PaymentMethod_Electronic check`, `InternetService_Fiber optic`) : ces deux variables complètent le top 6 des variables les plus influentes. Le paiement par chèque électronique ressort comme davantage associé au churn que les modes de paiement automatisés, plus stables, tandis que l'offre fibre est associée à une propension au churn plus élevée que les autres types d'accès.
- **Services de sécurité et de support** (`OnlineSecurity`, `TechSupport`) : bien que positionnés plus bas dans le classement d'importance (scores inférieurs à 0,025), l'absence de ces services additionnels reste associée à un taux de churn plus élevé, ce que confirme également l'analyse SHAP menée sur le meilleur modèle.

En revanche, les variables purement démographiques telles que `gender`, `SeniorCitizen`, `Partner` ou `Dependents` apparaissent nettement moins déterminantes : elles occupent les dernières positions du classement d'importance (scores inférieurs à 0,025), et l'analyse SHAP des interactions montre des valeurs d'impact resserrées autour de zéro pour `gender` et `SeniorCitizen`, confirmant qu'elles jouent un rôle marginal dans la décision de résiliation comparées aux variables contractuelles et financières.

3.3 Synthèse générale

L'ensemble de ces résultats dessine un profil-type du client à risque : un client récent, engagé sur un contrat sans engagement (mensuel), payant des charges mensuelles élevées, ne disposant pas de services de sécurité ou de support technique, et réglant ses factures par chèque électronique. Ce profil constitue la base des recommandations métier détaillées au chapitre suivant, et peut directement alimenter un score de risque opérationnel destiné aux équipes de rétention.

Chapitre 4

Recommandations métier

Sur la base des résultats obtenus au chapitre 3 — classement des variables par importance, corrélations et performances comparées des modèles — les recommandations suivantes peuvent être formulées à destination des équipes marketing et relation client, classées par ordre décroissant d'impact attendu.

R1. Renforcer l'accompagnement des nouveaux clients — `tenure` est de loin la variable la plus déterminante du modèle (score d'importance ~0,185, corrélation de $-0,35$ avec le churn), et le risque de résiliation est concentré sur les 5 premiers mois de la relation. Un parcours d'*onboarding* renforcé (suivi proactif, contacts de bienvenue, offres de fidélisation précoce) devrait être déployé en priorité sur cette fenêtre critique.

R2. Encourager les engagements longue durée — la modalité `Contract_Two year` figure au 4e rang des variables les plus importantes, confirmant son fort effet protecteur. Proposer des incitations tarifaires ou des avantages exclusifs aux clients en contrat mensuel qui acceptent de basculer vers un engagement d'un an ou deux ans, en priorisant les clients récents à charges mensuelles élevées.

R3. Revoir la politique tarifaire pour les clients à forte facture — `TotalCharges` et `MonthlyCharges` occupent les 2e et 3e rangs d'importance (~0,125 et ~0,09), et les churneurs paient en médiane 79 $/mois contre 64 $ pour les clients fidèles. Étudier des offres tarifaires adaptées (paliers, remises de fidélité) pour les clients dont la facture mensuelle dépasse significativement la médiane.

R4. Sécuriser les moyens de paiement — `PaymentMethod_Electronic check` est la 5e variable la plus importante du modèle. Accompagner activement les clients payant par chèque électronique vers des modes de paiement automatisés (prélèvement, carte bancaire), plus stables et moins associés au churn.

R5. Auditer l'offre fibre optique — `InternetService_Fiber optic` figure au 6e rang d'importance, suggérant que les clients fibre résilient davantage que les autres. Analyser la qualité perçue du service (débit, support technique, tarification) pour cette offre et envisager des actions correctives ciblées.

R6. Valoriser les services de sécurité et de support technique — bien que d'importance plus modeste (scores inférieurs à 0,025), `OnlineSecurity` et `TechSupport` restent associés à une meilleure rétention. Promouvoir activement ces offres auprès des clients qui ne les ont pas souscrites, en particulier lors du parcours d'onboarding.

R7. Déployer un score de risque opérationnel — intégrer le modèle **Random Forest**, retenu pour son meilleur score ROC-AUC (0,840) parmi les quatre modèles

testés, dans les outils CRM afin de cibler proactivement les clients à haut risque de churn avec des offres de rétention personnalisées.

Chapitre 5

Conclusion

Ce projet a permis de mettre en œuvre une chaîne complète de *Data Science*, depuis la compréhension et le nettoyage des données brutes jusqu'à la construction, la comparaison et l'interprétation de modèles de classification. La démarche a suivi sept objectifs clairement définis, couvrant l'exploration des données, la préparation rigoureuse du jeu d'entraînement, la modélisation par quatre algorithmes distincts, et l'interprétation fine des résultats via l'importance des variables et l'analyse SHAP.

Au-delà de la performance prédictive, l'apport principal de ce travail réside dans sa capacité à **transformer un problème de Machine Learning en leviers d'action concrets** pour les équipes de rétention client, illustrant ainsi la valeur métier d'une démarche de *Data Science* bien menée.